

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



SOICT

BÁO CÁO BÀI TẬP LỚN

Học phần: Project 1

Mã học phần: IT3150

Mã lớp: 755568

Đề tài:

ỨNG DỤNG CHATBOT
TRONG PHẦN MỀM QUẢN LÝ CHUNG CƯ

Giảng viên hướng dẫn: TS. Vũ Tuyết Trinh

Nhóm sinh viên thực hiện	Nguyễn Quang Huy	20230040
	Mai Lê Phú Quang	20230058

Hà Nội, Ngày 8 tháng 2 năm 2026

Mục lục

Lời nói đầu	4
1 Giới Thiệu	5
1.1 Tổng Quan Hệ Thống	5
1.1.1 Khả Năng Chính	5
1.1.2 Ngăn Xếp Công Nghệ	5
1.2 Kiến Trúc Hệ Thống	6
1.2.1 Tổng Quan Kiến Trúc	6
1.2.2 Luồng Xử Lý Trong ChatService	6
1.2.3 Tích Hợp Với Các Hệ Thống Bên Ngoài	7
1.3 Đặc Điểm Kỹ Thuật Nổi Bật	7
1.3.1 Sinh SQL Hai Giai Đoạn	7
1.3.2 Cơ Chế Tự Sửa Lỗi	8
1.3.3 Nội Quan Schema Động	8
1.3.4 Hội Thoại Nhận Biết Ngữ Cảnh	9
1.4 Bảo Mật Và Phân Quyền	9
1.4.1 Kiểm Soát Truy Cập Dựa Trên Vai Trò	9
1.4.2 Ngăn Chặn SQL Injection	9
2 Cơ Sở Lý Thuyết	11
2.1 Spring Boot Framework	11
2.1.1 Các Thành Phần Quan Trọng	11
2.2 Xử Lý Ngôn Ngữ Tự Nhiên (NLP)	12
2.2.1 Khái Niệm NLP	12
2.2.2 Các Kỹ Thuật NLP Trong Dự Án	12
2.2.3 Large Language Models (LLMs)	12
2.3 Text-to-SQL	13
2.3.1 Định Nghĩa Bài Toán	13
2.3.2 Các Thách Thức	13
2.3.3 Phương Pháp Hai Giai Đoạn	13
2.3.4 Schema Linking	14
2.4 Prompt Engineering	14
2.4.1 Định Nghĩa	14
2.4.2 Các Thành Phần Của Prompt	15
2.4.3 Few-Shot Learning	15
2.4.4 Chain-of-Thought Prompting	15
2.4.5 Constraint Specification	16
2.5 Bảo Mật và Phân Quyền	16
2.5.1 Spring Security	16
2.5.2 Role-Based Access Control (RBAC)	16
2.5.3 SQL Injection Prevention	17
2.6 Session Management	17
2.6.1 HTTP Sessions	17
2.6.2 Spring Session Scopes	18
2.7 RESTful API Design	18
2.7.1 REST Principles	18

2.7.2	HTTP Methods	19
2.7.3	HTTP Status Codes	19
2.7.4	JSON Format	19
2.8	Kết Luận	20
3	Các Thành Phần Hệ Thống	21
3.1	Tổng Quan Kiến Trúc	21
3.1.1	Mô Hình Phân Lớp	21
3.2	ChatController	21
3.2.1	Chức Năng	21
3.2.2	API Endpoints	21
3.2.3	Cấu Hình	22
3.3	ChatService	22
3.3.1	Cấu Hình Scope	22
3.3.2	Dependencies	23
3.3.3	State Management	23
3.4	Hệ Thống Bảo Mật và Phân Quyền	23
3.4.1	Authorization Logic	23
3.4.2	Enforcement	23
3.5	Pipeline Xử Lý Tin Nhắn	25
3.5.1	Bước 1: Authorization Check	25
3.5.2	Bước 2: Intent Classification	25
3.5.3	Bước 3: SQL Generation	25
3.5.4	Bước 4: Query Execution	26
3.5.5	Bước 5: Result Summarization	26
3.6	Nội Quan Schema Động	26
3.6.1	Initialization	26
3.6.2	Extraction Process	26
3.6.3	Helper Methods	26
3.6.4	Generated Schema Context	27
3.7	Quản Lý Hội Thoại	27
3.7.1	Conversation History	27
3.7.2	Context Management	27
3.7.3	Lợi Ích	27
3.8	Tích Hợp AI	27
3.8.1	Groq API Client	27
3.8.2	Temperature Settings	28
3.9	Semantic Mapping	28
3.9.1	Vietnamese-to-SQL Mapping	28
3.9.2	Evidence String	28
3.10	Xử Lý Lỗi	28
3.10.1	Error Types	28
3.10.2	Error Recovery	29
3.11	Performance Optimization	29
3.11.1	Schema Caching	29
3.11.2	Result Limiting	29
3.11.3	Connection Pooling	29
3.11.4	Conversation Window	29
3.12	Configuration Management	29
3.12.1	External Configuration	29
3.12.2	Application Properties	29
3.12.3	Environment Variables	30
3.13	Data Transfer Objects	30
3.13.1	ChatRequestDto	30
3.13.2	ChatResponse	30
3.14	Tương Tác Giữa Các Thành Phần	30
3.14.1	Request Flow	30
3.14.2	Component Dependencies	31

3.15	Tóm Tắt	31
4	Cài đặt chương trình	32
4.1	Cấu trúc dự án	32
4.1.1	Backend (Spring Boot)	32
4.1.2	Frontend (React)	32
4.2	Cài đặt tính năng Chatbot AI	32
4.2.1	Logic xử lý (ChatService.java)	32
4.2.2	Giao diện Chatbot (Chatbot.js)	33
5	Đánh Giá và Phân Tích Kết Quả	34
5.1	Phân Tích Điểm Mạnh - Điểm Yếu	34
5.2	Kết Quả Đạt Được	34
5.2.1	Về mặt Chức năng	34
5.2.2	Về mặt Kỹ thuật	34
6	Kết luận và Hướng phát triển	35
6.1	Kết luận	35
6.2	Hạn chế	35
6.3	Hướng phát triển	35
6.4	Phân công nhiệm vụ	36

Lời nói đầu

Trong kỷ nguyên Chuyển đổi số (Digital Transformation), dữ liệu đã trở thành tài sản quý giá nhất của mọi tổ chức. Tuy nhiên, việc truy xuất và khai thác dữ liệu này thường đòi hỏi kiến thức chuyên môn về ngôn ngữ truy vấn (SQL) hoặc phụ thuộc vào các báo cáo tính định kỳ. Điều này tạo ra một rào cản lớn đối với người dùng cuối – những cư dân hay ban quản lý chung cư, những người mong muốn có được thông tin tức thời thông qua ngôn ngữ tự nhiên hàng ngày.

Sự bùng nổ của các Mô hình Ngôn ngữ Lớn (Large Language Models - LLMs) như GPT-4 hay LLaMA đã mở ra một hướng đi mới cho việc tương tác giữa người và máy. Khả năng hiểu ngữ nghĩa và sinh văn bản của LLM, khi kết hợp với cấu trúc chặt chẽ của Cơ sở dữ liệu quan hệ (RDBMS), có thể tạo ra những trợ lý ảo thông minh, giúp xóa bỏ khoảng cách kỹ thuật trong việc quản lý thông tin.

Xuất phát từ thực tế đó, nhóm chúng tôi đã lựa chọn đề tài "**Xây dựng Ứng dụng Quản lý Chung cư tích hợp Chatbot AI (ApartmentAI)**". Mục tiêu của dự án là xây dựng một hệ thống backend vững chắc dựa trên Spring Boot, kết hợp với sức mạnh của Groq API để cung cấp khả năng *Text-to-SQL* – chuyển đổi câu hỏi tiếng Việt thành câu lệnh truy vấn dữ liệu chính xác, đồng thời đảm bảo tính bảo mật và phân quyền nghiêm ngặt.

Dự án **ApartmentAI** không chỉ là một bài tập kỹ thuật, mà còn là nỗ lực của nhóm trong việc áp dụng các công nghệ tiên tiến nhất hiện nay (Spring Boot 3, ReactJS, GenAI) để giải quyết bài toán nghiệp vụ thực tế. Chúng tôi tập trung vào việc xử lý các thách thức về ngữ cảnh hội thoại, tự động sửa lỗi truy vấn (Self-healing) và cơ chế phân quyền động (Dynamic RBAC) trong môi trường AI.

Để hoàn thành đồ án này, chúng em xin gửi lời cảm ơn chân thành tới **TS. Vũ Tuyết Trinh**, người đã tận tình hướng dẫn, định hướng và đưa ra những nhận xét quý báu giúp chúng em hoàn thiện sản phẩm. Đồng thời, chúng em xin cảm ơn **Trường Công nghệ Thông tin và Truyền thông - ĐHBách Khoa Hà Nội** đã trang bị nền tảng kiến thức vững chắc để chúng em thực hiện đề tài này.

Bộ cục báo cáo gồm 6 chương:

- **Chương 1 - Giới thiệu:** Tổng quan về bài toán quản lý chung cư và kiến trúc tổng thể của hệ thống ApartmentAI.
- **Chương 2 - Cơ sở lý thuyết:** Trình bày về Spring Boot, công nghệ NLP/LLM, kỹ thuật Prompt Engineering và Text-to-SQL.
- **Chương 3 - Các thành phần hệ thống:** Đi sâu vào thiết kế ChatController, ChatService và cơ chế bảo mật RBAC tích hợp AI.
- **Chương 4 - Cài đặt chương trình:** Mô tả cấu trúc mã nguồn, các lớp thực thi chính và tích hợp API.
- **Chương 5 - Kết luận và đánh giá:** Phân tích điểm mạnh, hạn chế và tiềm năng thực tế của hệ thống.
- **Chương 6 - Hướng phát triển:** Đề xuất các cải tiến về Mobile App và AI nâng cao.

Link Github của project: <https://github.com/quangmlp/ApartmentAI>

Nhóm sinh viên thực hiện

Chương 1

Giới Thiệu

1.1 Tổng Quan Hệ Thống

Dịch vụ Chatbot ApartmentAI là một hệ thống trí tuệ nhân tạo hội thoại tiên tiến, cho phép tương tác bằng ngôn ngữ tự nhiên với dữ liệu quản lý chung cư. Được xây dựng trên nền tảng Spring Boot, hệ thống kết hợp các khả năng AI hiện đại với các biện pháp bảo mật mạnh mẽ để cung cấp các phản hồi thông minh, nhận biết ngữ cảnh, đồng thời thực thi kiểm soát truy cập dữ liệu nghiêm ngặt.



Hình 1.1: Mô hình ngữ cảnh hệ thống ApartmentAI

1.1.1 Khả Năng Chính

Hệ thống chatbot cung cấp các khả năng sau:

- Chuyển đổi ngôn ngữ tự nhiên sang SQL với hỗ trợ tiếng Việt
- Kiểm soát truy cập dữ liệu dựa trên vai trò (Admin và Cư dân)
- Tự động sửa lỗi trong quá trình sinh truy vấn
- Hội thoại đa lượt với ngữ cảnh liên tục
- Tự động thích ứng với cấu trúc cơ sở dữ liệu

1.1.2 Ngăn Xếp Công Nghệ

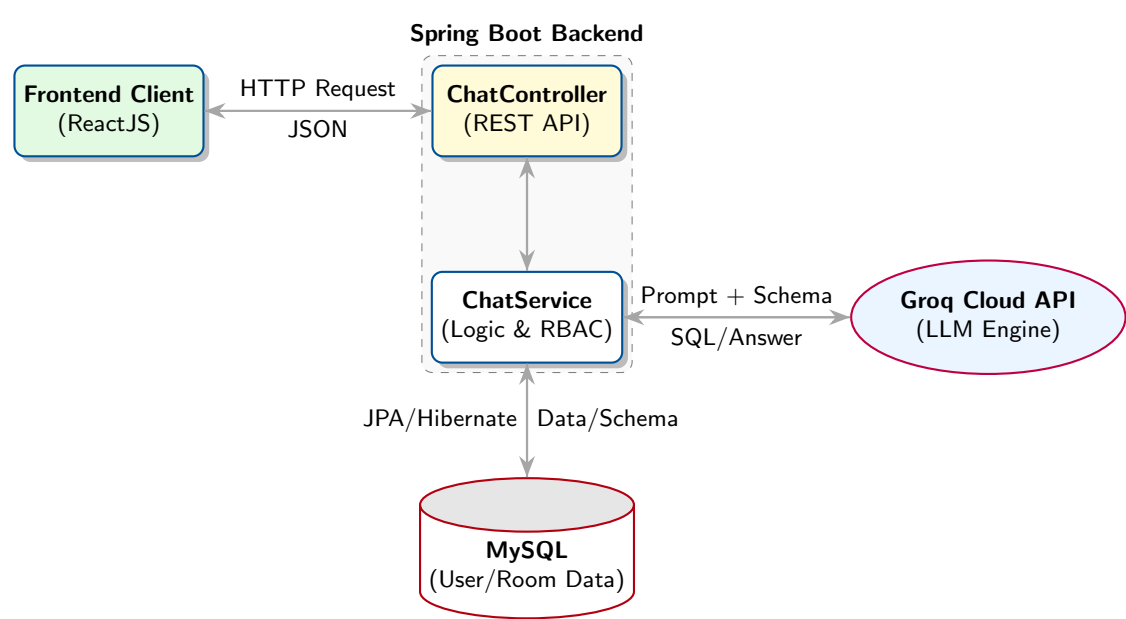
Hệ thống được xây dựng dựa trên các công nghệ sau:

Hạng mục	Công nghệ	Vai trò
Backend Framework	Spring Boot 3.x	Xây dựng REST API, quản lý logic nghiệp vụ và bảo mật.
AI Engine	Groq Cloud API	Cung cấp mô hình LLM (Llama3/Mixtral) tốc độ cao để xử lý NLP.
Database	MySQL	Lưu trữ dữ liệu quan hệ (Cư dân, Phòng, Phí).
ORM	Hibernate / JPA	Ánh xạ đối tượng - quan hệ, thao tác dữ liệu.
Frontend	ReactJS	Xây dựng giao diện người dùng tương tác.

Bảng 1.1: Danh sách công nghệ sử dụng trong dự án

1.2 Kiến Trúc Hệ Thống

1.2.1 Tổng Quan Kiến Trúc



Hình 1.2: Sơ đồ kiến trúc tổng thể hệ thống ApartmentAI

Hệ thống được thiết kế theo kiến trúc phân lớp với các thành phần chính sau:

- Lớp Frontend:** Ứng dụng React.js, giao tiếp với backend qua HTTP/REST
- Lớp ChatController:** Điều khiển REST, xử lý routing và CORS
- Lớp ChatService:** Lớp logic nghiệp vụ, xử lý các chức năng chính
- Lớp Dữ Liệu:** Cơ sở dữ liệu MySQL chứa thông tin phòng, người dùng, xe và phí
- Lớp AI:** Groq API cung cấp khả năng xử lý ngôn ngữ tự nhiên

1.2.2 Luồng Xử Lý Trong ChatService

Lớp ChatService thực hiện quy trình xử lý phức tạp gồm 5 bước chính:

- Kiểm Tra Phân Quyền (RBAC):** Xác định quyền truy cập của người dùng dựa trên vai trò và quyền sở hữu phòng
- Phân Loại Ý Định:** Phân biệt giữa hội thoại thường (chào hỏi, cảm ơn) và truy vấn dữ liệu

3. **Pipeline Sinh SQL:** Quá trình sinh SQL gồm nhiều giai đoạn:

- *Composer*: Phân tách câu hỏi phức tạp thành các câu hỏi con
- *Generator*: Tạo truy vấn SQL từ câu hỏi
- *Validator*: Chạy thử nghiệm (dry run) để kiểm tra lỗi
- *Refiner*: Tự động sửa lỗi dựa trên phản hồi từ database

4. **Thực Thi Truy Vấn:** Chạy SQL trên cơ sở dữ liệu MySQL

5. **Tóm Tắt Kết Quả:** Chuyển đổi kết quả SQL thành câu trả lời tự nhiên bằng tiếng Việt

1.2.3 Tích Hợp Với Các Hệ Thống Bên Ngoài

Hệ thống tương tác với hai hệ thống bên ngoài chính:

1.2.3.1 Groq API (Mô Hình AI)

Groq API cung cấp khả năng xử lý ngôn ngữ tự nhiên với các chức năng:

- Phân loại ý định người dùng
- Sinh truy vấn SQL từ câu hỏi tiếng Việt
- Tự động sửa lỗi SQL
- Tóm tắt kết quả thành ngôn ngữ tự nhiên

Mô hình sử dụng: `openai/gpt-oss-120b` - một mô hình GPT mã nguồn mở với 120 tỷ tham số.

1.2.3.2 Cơ Sở Dữ Liệu MySQL

Database lưu trữ thông tin quản lý chung cư:

- **Bảng rooms:** Thông tin phòng (số phòng, diện tích, số người)
- **Bảng users:** Thông tin người dùng và vai trò
- **Bảng vehicles:** Thông tin phương tiện (xe máy, ô tô)
- **Bảng fees:** Thông tin phí và đóng góp

1.3 Đặc Điểm Kỹ Thuật Nổi Bật

1.3.1 Sinh SQL Hai Giai Đoạn

Hệ thống sử dụng phương pháp sinh SQL hai giai đoạn được lấy cảm hứng từ nghiên cứu học thuật về text-to-SQL:

1.3.1.1 Giai Đoạn 1: Composer (Phân Tách)

Giai đoạn này phân tách câu hỏi phức tạp thành các câu hỏi con logic:

- Nhận câu hỏi ngôn ngữ tự nhiên
- Cung cấp ngữ cảnh schema database
- Áp dụng quy tắc ánh xạ ngữ nghĩa (Tiếng Việt → SQL)
- Yêu cầu phân tách từng bước
- Trích xuất SQL cuối cùng từ output có cấu trúc

Ví dụ Output:

Câu hỏi con 1: Có bao nhiêu phòng?

```
SQL: SELECT COUNT(*) FROM rooms;
```

Câu hỏi con 2: Phòng nào có xe máy?

```
SQL: SELECT DISTINCT r.room_number
      FROM rooms r
      JOIN vehicles v ON r.id = v.room_id
      WHERE v.type = 'MOTORBIKE';
```

1.3.1.2 Giai Đoạn 2: Refiner (Tinh Chỉnh)

Giai đoạn này tự động sửa lỗi SQL thông qua phản hồi từ AI:

- Bắt thông báo lỗi và loại exception
- Gửi ngữ cảnh lỗi về AI
- Yêu cầu SQL đã được sửa
- Trả về truy vấn đã được khắc phục

1.3.2 Cơ Chế Tự Sửa Lỗi

Hệ thống tự động thử sửa các truy vấn thất bại mà không cần can thiệp của người dùng:

1. **Lần thử 1:** Sinh SQL từ câu hỏi người dùng
2. **Dry Run:** Thực thi SQL để kiểm tra lỗi
3. **Nếu thành công:** Trả về SQL
4. **Nếu thất bại:**
 - Bắt chi tiết lỗi
 - Gửi lỗi về AI với yêu cầu sửa
 - Nhận SQL đã được sửa
 - Trả về SQL cuối cùng

1.3.3 Nội Quan Schema Động

Hệ thống tự động trích xuất và duy trì metadata của database schema:

- Sử dụng JPA EntityManager để lấy metadata
- Trích xuất tên bảng vật lý từ annotation @Table
- Trích xuất tên cột vật lý từ annotation @Column
- Xử lý đặc biệt cho Enum (liệt kê tất cả giá trị)
- Nhận diện bảng quan hệ (@ElementCollection)

Lợi ích:

- Không cần tài liệu schema thủ công
- Tự động thích ứng với thay đổi schema
- Cung cấp ngữ cảnh chính xác cho mô hình AI

1.3.4 Hội Thoại Nhận Biết Ngữ Cảnh

Hệ thống duy trì lịch sử hội thoại để xử lý các cuộc đối thoại đa lượt:

- Lưu trữ tối đa 10 tin nhắn gần nhất (rolling window)
- Bao gồm cả tin nhắn của người dùng và trợ lý
- Phạm vi session (cô lập theo người dùng)
- Sử dụng cho ngữ cảnh trong các câu hỏi tiếp theo

Lợi ích:

- Câu hỏi tiếp theo: “Còn phòng 102 thì sao?”
- Phân giải đại từ: “Có bao nhiêu xe?”
- Liên tục ngữ cảnh: “Hiện thị phí của họ”

1.4 Bảo Mật Và Phân Quyền

1.4.1 Kiểm Soát Truy Cập Dựa Trên Vai Trò

Chatbot triển khai logic phân quyền tinh vi để hạn chế truy cập dữ liệu dựa trên vai trò người dùng.

1.4.1.1 Ma Trận Phân Quyền

Loại Người Dùng	Quyền	Hạn Chế SQL
Admin	Truy cập đầy đủ	Không hạn chế
Cư dân (có phòng)	Chỉ truy cập phòng của mình	WHERE room_number IN ('101', '102', ...)
Cư dân (không có phòng)	Không thể truy vấn dữ liệu chung cư	Trả về NOT_AUTHORIZED
Ấn danh	Chỉ thông tin công khai	Không có dữ liệu nhạy cảm

Bảng 1.2: Ma trận phân quyền người dùng

1.4.1.2 Thực Thi Bảo Mật

Hệ thống thực thi bảo mật qua các cơ chế sau:

- **Injection vào AI Prompt:** Ngữ cảnh hạn chế được đưa vào prompt của AI
- **Lọc Ở Cấp SQL:** Mô hình AI thực thi lọc dữ liệu khi sinh SQL
- **Ngăn Chặn Truy Cập Không Hợp Lệ:** Các nỗ lực truy cập không hợp lệ trả về NOT_AUTHORIZED
- **Không Lọc Sau Truy Vấn:** Ngăn chặn ngay từ nguồn, không cần lọc sau khi query

1.4.2 Ngăn Chặn SQL Injection

Hệ thống triển khai nhiều lớp bảo vệ chống SQL injection:

1. **Xác Thực Loại Truy Vấn:** Chỉ cho phép câu lệnh SELECT
2. **Tham Số Được Kiểm Soát:** Native query với tham số được kiểm soát
3. **SQL Do AI Sinh:** SQL không được tạo trực tiếp từ input người dùng
4. **Cô Lập Session:** Mỗi session có ngữ cảnh hội thoại riêng

Code kiểm tra bảo mật:

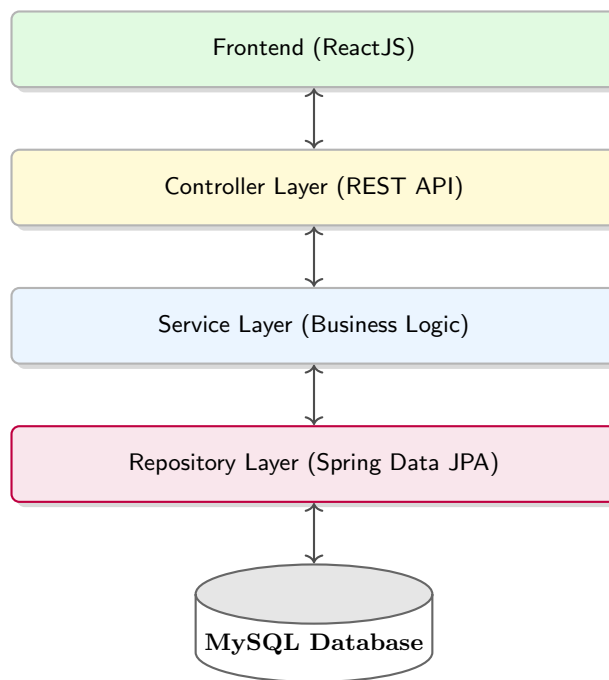
```
if (!sql.trim().toUpperCase().startsWith("SELECT")) {  
    throw new IllegalArgumentException("Unauthorized SQL");  
}
```

Chương 2

Cơ Sở Lý Thuyết

2.1 Spring Boot Framework

Spring Boot là framework Java được sử dụng để xây dựng backend của hệ thống, cung cấp auto-configuration, embedded server, và dependency injection.



Hình 2.1: Kiến trúc phân lớp trong Spring Boot

2.1.1 Các Thành Phần Quan Trọng

- **Spring MVC:** Xử lý HTTP requests và REST API (`@RestController`, `@RequestMapping`)
- **Dependency Injection:** Quản lý dependencies với `@Autowired`, `@Service`, `@Component`
- **Spring Data JPA:** Truy cập database với `EntityManager`, repositories, và annotations (`@Entity`, `@Table`, `@Column`)
- **Session Management:** `@SessionScope` cho `ChatService` để cô lập conversation per user

2.2 Xử Lý Ngôn Ngữ Tự Nhiên (NLP)

2.2.1 Khái Niệm NLP

Natural Language Processing (NLP) là lĩnh vực của AI tập trung vào việc cho phép máy tính hiểu, diễn giải và tạo ra ngôn ngữ tự nhiên của con người. NLP kết hợp giữa computational linguistics và machine learning.

2.2.2 Các Kỹ Thuật NLP Trong Dự Án

2.2.2.1 1. Intent Classification

Intent classification là quá trình xác định ý định của người dùng từ input text.

Phương pháp:

- Sử dụng AI model để phân loại
- Training với examples của các intent types
- Output: TRUE (chit-chat) hoặc FALSE (data query)

Ví dụ:

- “Xin chào” → Chit-chat (TRUE)
- “Có bao nhiêu phòng?” → Data query (FALSE)

2.2.2.2 2. Semantic Parsing

Semantic parsing chuyển đổi natural language thành formal representations (SQL).

Các bước:

1. **Tokenization:** Tách câu thành các tokens
2. **Entity Recognition:** Nhận diện entities (phòng, xe, phí)
3. **Semantic Mapping:** Ánh xạ từ tiếng Việt sang SQL concepts
4. **Query Generation:** Sinh SQL từ semantic representation

2.2.2.3 3. Named Entity Recognition (NER)

NER nhận diện và phân loại named entities trong text.

Entities trong ApartmentAI:

- Room numbers: “phòng 101”, “căn 102”
- Vehicle types: “xe máy”, “ô tô”
- Quantities: “bao nhiêu”, “tổng cộng”
- Time expressions: “tháng này”, “năm ngoái”

2.2.3 Large Language Models (LLMs)

Large Language Models là các mô hình AI được train trên lượng lớn text data để hiểu và sinh ngôn ngữ tự nhiên. Dự án sử dụng Groq API với model openai/gpt-oss-120b.

2.2.3.1 Temperature Parameter

Temperature điều khiển randomness của model output:

$$P(x_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}} \quad (2.1)$$

Trong đó: T = temperature, z_i = logit cho token i , $P(x_i)$ = probability của token i

Tác vụ (Task)	Temp (T)	Lý do thiết lập
Sinh mã SQL	0.1	Cần độ chính xác tuyệt đối, deterministic, tránh ảo giác (hallucination).
Phân loại ý định	0.1	Cần kết quả phân loại (True/False) nhất quán.
Tóm tắt kết quả	0.5	Cân bằng giữa sự chính xác của dữ liệu và văn phong tự nhiên.
Hội thoại thường (Chit-chat)	0.7	Tăng tính sáng tạo, thân thiện và đa dạng trong câu trả lời.

Bảng 2.1: Thiết lập tham số Temperature cho các tác vụ khác nhau

2.3 Text-to-SQL

2.3.1 Định Nghĩa Bài Toán

Text-to-SQL là bài toán chuyển đổi câu hỏi ngôn ngữ tự nhiên thành SQL queries có thể thực thi.

Input: Natural language question + Database schema

Output: SQL query

Ví dụ:

Mẫu Prompt Few-Shot (Minh họa)

Instruction: Convert Vietnamese questions to SQL based on the schema.

Example:

User: "Có bao nhiêu phòng?"

SQL: SELECT COUNT(*) FROM rooms;

2.3.2 Các Thách Thức

- Schema Complexity:** Databases có nhiều tables và relationships phức tạp
- Ambiguity:** Câu hỏi tự nhiên có thể có nhiều cách hiểu
- Implicit Information:** Thông tin không được nói rõ trong câu hỏi
- Complex Queries:** Joins, aggregations, subqueries
- Domain-Specific Vocabulary:** Thuật ngữ chuyên ngành

2.3.3 Phương Pháp Hai Giai Đoạn

Dự án ApartmentAI sử dụng phương pháp hai giai đoạn để cải thiện độ chính xác.

2.3.3.1 Giai Đoạn 1: Decomposition

Phân tách câu hỏi phức tạp thành các câu hỏi con đơn giản.

Lý thuyết: Complex reasoning có thể được chia thành simple reasoning steps (Chain-of-Thought).

Ví dụ:

- Question: "Phòng nào có nhiều xe nhất?"

- Sub-question 1: “Mỗi phòng có bao nhiêu xe?”
- Sub-question 2: “Phòng nào có số lượng xe lớn nhất?”

Lợi ích:

- Dễ sinh SQL cho câu hỏi đơn giản
- Model có thể reasoning step-by-step
- Giảm errors trong complex queries

2.3.3.2 Giai Đoạn 2: Refinement

Tự động sửa lỗi SQL dựa trên error feedback.

Lý thuyết: Self-correction through iterative refinement.

Process:

1. Execute generated SQL
2. Nếu có lỗi, capture error message
3. Gửi error + old SQL về model
4. Model sinh corrected SQL

Error Types:

- **Syntax Error:** SQL không hợp lệ
- **Schema Error:** Table/column không tồn tại
- **Type Error:** Data type mismatch
- **Logic Error:** SQL đúng cú pháp nhưng logic sai

2.3.4 Schema Linking

Schema linking là quá trình map các mentions trong question với elements trong schema.

Ví dụ:

- “xe máy” → vehicles.type = 'MOTORBIKE'
- “số phòng” → rooms.room_number
- “có bao nhiêu” → COUNT(*)

Kỹ thuật:

- **Exact Match:** String matching trực tiếp
- **Fuzzy Match:** Similar strings (edit distance)
- **Semantic Match:** Similar meaning (embeddings)
- **Predefined Rules:** Domain-specific mappings

2.4 Prompt Engineering**2.4.1 Định Nghĩa**

Prompt engineering là nghệ thuật thiết kế inputs (prompts) cho AI models để đạt được desired outputs.

2.4.2 Các Thành Phần Của Prompt

1. **Instruction:** Mô tả task cần thực hiện
2. **Context:** Thông tin background (schema, rules)
3. **Input Data:** Dữ liệu cụ thể cần xử lý
4. **Output Format:** Định dạng output mong muốn

2.4.3 Few-Shot Learning

Few-shot learning là kỹ thuật cung cấp examples trong prompt để guide model behavior.

Format:

Instruction: Convert questions to SQL

Example 1:

Q: How many rooms?

SQL: SELECT COUNT(*) FROM rooms

Example 2:

Q: List all vehicles

SQL: SELECT * FROM vehicles

Now do:

Q: [User's question]

SQL:

2.4.4 Chain-of-Thought Prompting

Chain-of-Thought (CoT) yêu cầu model explain reasoning steps trước khi output final answer.

Format:

Question: [Complex question]

Think step by step:

Step 1: [First reasoning step]

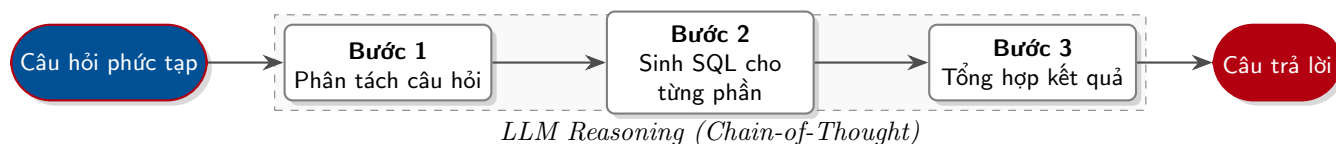
Step 2: [Second reasoning step]

...

Final SQL: [Generated query]

Lợi ích:

- Cải thiện accuracy cho complex reasoning
- Dễ debug khi có lỗi
- Model tự kiểm tra logic



Hình 2.2: Mô hình xử lý Chain-of-Thought trong LLM

2.4.5 Constraint Specification

Chỉ định constraints trong prompt để guide model behavior.

Ví dụ trong ApartmentAI:

- “Only select needed columns”
- “Avoid unnecessary JOINS”
- “Use GROUP BY before ORDER BY”
- “Handle NULL values properly”

2.5 Bảo Mật và Phân Quyền

2.5.1 Spring Security

Spring Security là framework bảo mật toàn diện cho Spring applications.

2.5.1.1 Authentication

Authentication là quá trình xác minh identity của user.

Các phương thức:

- **Form-based:** Username/password qua form
- **HTTP Basic:** Credentials trong HTTP header
- **JWT:** JSON Web Tokens
- **OAuth2:** Delegated authentication

SecurityContext:

- Lưu trữ authentication information
- Thread-local storage
- Accessible qua `SecurityContextHolder`

2.5.1.2 Authorization

Authorization là quá trình kiểm tra permissions của authenticated user.

Các mức authorization:

1. **URL-level:** Restrict access đến endpoints
2. **Method-level:** Restrict method execution
3. **Object-level:** Restrict access đến specific objects
4. **Field-level:** Restrict access đến specific fields

2.5.2 Role-Based Access Control (RBAC)

RBAC là mô hình phân quyền dựa trên roles của users.

2.5.2.1 Các Thành Phần

- **User:** Entities cần authentication và authorization
- **Role:** Tập hợp permissions (VD: ADMIN, USER, RESIDENT)
- **Permission:** Specific actions (VD: READ, WRITE, DELETE)
- **Resource:** Objects cần protection (VD: rooms, vehicles)

Role	Read All	Read Own	Write	Delete
ADMIN	v	x	x	v
RESIDENT	x	v	x	x
GUEST	x	x	x	x

Bảng 2.2: Permissions theo role

2.5.2.2 RBAC Matrix

2.5.2.3 Dynamic Authorization

Trong ApartmentAI, authorization được thực thi động dựa trên:

- Role của user (ADMIN vs RESIDENT)
- Room ownership (user sở hữu phòng nào)
- Query context (đang query về phòng nào)

Implementation:

1. Get authenticated user từ SecurityContext
2. Check user's roles
3. Nếu không phải ADMIN:
 - Query user's rooms
 - Build restriction context
 - Inject vào SQL generation prompt
4. AI model enforce restrictions khi sinh SQL

2.5.3 SQL Injection Prevention

SQL injection là vulnerability cho phép attackers execute arbitrary SQL.

2.5.3.1 Các Phương Pháp Phòng Chống

1. **Parameterized Queries:** Sử dụng prepared statements
2. **Input Validation:** Validate và sanitize inputs
3. **Least Privilege:** Database user có minimum permissions
4. **Query Type Restriction:** Chỉ cho phép SELECT queries
5. **Whitelisting:** Chỉ accept known-good inputs

2.5.3.2 Trong ApartmentAI

- SQL được sinh bởi AI, không trực tiếp từ user input
- Validate SQL starts with SELECT
- Không allow INSERT/UPDATE/DELETE/DROP
- Native query execution với controlled parameters

2.6 Session Management

2.6.1 HTTP Sessions

HTTP là stateless protocol, sessions được sử dụng để maintain state across requests.

2.6.1.1 Session Lifecycle

1. **Creation:** Session được tạo khi user gửi first request
2. **Identification:** Session ID được gửi qua cookie hoặc URL
3. **Usage:** Server lookup session data dựa trên session ID
4. **Expiration:** Session expires sau timeout period
5. **Invalidation:** Session bị invalidate khi user logout

2.6.1.2 Session Storage

Options:

- **In-Memory:** Lưu trong RAM (mặc định)
- **Cookie-based:** Lưu trong client cookies
- **Database:** Persist vào database
- **Redis:** Distributed cache

2.6.2 Spring Session Scopes

Spring cung cấp nhiều bean scopes để quản lý lifecycle.

2.6.2.1 Các Scopes

- **Singleton:** Một instance cho toàn application (mặc định)
- **Prototype:** New instance mỗi lần inject
- **Request:** Một instance per HTTP request
- **Session:** Một instance per HTTP session
- **Application:** Một instance per ServletContext

2.6.2.2 SessionScope trong ApartmentAI

ChatService được config với @SessionScope:

Lợi ích:

- Mỗi user có ChatService instance riêng
- Conversation history isolated per user
- Không có data leakage giữa users
- Automatic cleanup khi session expires

Lưu ý:

- Memory usage tăng theo số sessions
- Cần config appropriate session timeout
- Không scale well với large user base (cần distributed session)

2.7 RESTful API Design

2.7.1 REST Principles

REST (Representational State Transfer) là architectural style cho distributed systems.

2.7.1.1 6 Constraints

1. **Client-Server:** Separation of concerns
2. **Stateless:** No client context stored on server
3. **Cacheable:** Responses có thể được cached
4. **Uniform Interface:** Standardized communication
5. **Layered System:** Hierarchical layers
6. **Code on Demand (optional):** Execute code from server

2.7.2 HTTP Methods

Method	Purpose	Idempotent	Safe
GET	Retrieve resource	Yes	Yes
POST	Create resource	No	No
PUT	Update/Replace resource	Yes	No
PATCH	Partial update	No	No
DELETE	Delete resource	Yes	No

Bảng 2.3: HTTP Methods

2.7.3 HTTP Status Codes

- **2xx Success:**
 - 200 OK: Request succeeded
 - 201 Created: Resource created
 - 204 No Content: Success, no body
- **4xx Client Errors:**
 - 400 Bad Request: Invalid request
 - 401 Unauthorized: Authentication required
 - 403 Forbidden: No permission
 - 404 Not Found: Resource doesn't exist
- **5xx Server Errors:**
 - 500 Internal Server Error: Server error
 - 503 Service Unavailable: Server overloaded

2.7.4 JSON Format

JSON (JavaScript Object Notation) là format phổ biến cho REST APIs.

Ưu điểm:

- Human-readable
- Lightweight
- Language-agnostic
- Native support trong JavaScript

Example request:

```
{  
  "message": "Có bao nhiêu phòng?"  
}
```

Example response:

```
{  
  "response": "Hiện có 120 phòng trong tòa nhà.",  
  "timestamp": "2026-01-04T10:30:00Z"  
}
```

2.8 Kết Luận

Các cơ sở lý thuyết được trình bày trong chương này cung cấp nền tảng vững chắc cho việc hiểu và phát triển hệ thống Chatbot ApartmentAI:

- **Spring Boot:** Framework mạnh mẽ cho enterprise Java applications
- **NLP và LLMs:** Công nghệ AI hiện đại cho natural language understanding
- **Text-to-SQL:** Phương pháp chuyển đổi language sang structured queries
- **Prompt Engineering:** Kỹ thuật tối ưu AI model performance
- **Security:** Bảo vệ dữ liệu và kiểm soát truy cập
- **REST API:** Standard cho web services communication

Sự kết hợp các công nghệ này tạo nên một hệ thống chatbot thông minh, bảo mật và có khả năng mở rộng cao.

Chương 3

Các Thành Phần Hệ Thống

3.1 Tổng Quan Kiến Trúc

Hệ thống Chatbot ApartmentAI được thiết kế theo kiến trúc phân lớp (Layered Architecture) với hai thành phần chính: ChatController và ChatService. Kiến trúc này đảm bảo separation of concerns, dễ bảo trì và mở rộng.

3.1.1 Mô Hình Phân Lớp

- **Presentation Layer:** ChatController - xử lý HTTP requests/responses
- **Business Logic Layer:** ChatService - xử lý logic nghiệp vụ
- **Data Access Layer:** Repositories và EntityManager - truy cập database
- **External Integration Layer:** Groq API client - tích hợp AI

3.2 ChatController

ChatController là REST controller chịu trách nhiệm expose API endpoints cho frontend.

3.2.1 Chức Năng

- Nhận HTTP requests từ client
- Validate và parse request body
- Gọi ChatService để xử lý business logic
- Format và trả về responses
- Xử lý CORS configuration

3.2.2 API Endpoints

Method	Endpoint	Chức năng	Mô tả
POST	/api/chat	Gửi tin nhắn	Nhận câu hỏi từ người dùng, trả về câu trả lời của AI. Payload: JSON chứa message .
GET	/api/chat/history	Lấy lịch sử	Lấy lại danh sách tin nhắn cũ của phiên làm việc hiện tại.
DELETE	/api/chat	Xóa phiên	Xóa ngữ cảnh hội thoại để bắt đầu phiên mới.

Bảng 3.1: Danh sách các API Endpoints của Chatbot

3.2.2.1 POST /chat

Endpoint chính để xử lý tin nhắn từ người dùng.

Request:

POST /chat

Content-Type: application/json

```
{
  "message": "Có bao nhiêu xe máy trong tòa nhà?"
}
```

Response:

HTTP 200 OK

Content-Type: application/json

```
{
  "response": "Hiện tại có 25 xe máy trong tòa nhà."
}
```

3.2.2.2 GET /chat

Health check endpoint để kiểm tra trạng thái service.

Response:

HTTP 200 OK

Content-Type: text/plain

"Chatbot service is running!"

3.2.3 Cấu Hình

- **@RestController:** Đánh dấu class là REST controller
- **@RequestMapping("/chat"):** Base URL cho tất cả endpoints
- **@CrossOrigin(origins = "*"):** Cho phép CORS từ mọi origin
- **@Autowired ChatService:** Inject ChatService dependency

3.3 ChatService

ChatService là core của hệ thống, chứa toàn bộ business logic và xử lý chatbot.

3.3.1 Cấu Hình Scope

ChatService được config với **@SessionScope**, nghĩa là mỗi user session có một instance riêng. Điều này đảm bảo:

- Conversation history riêng biệt cho mỗi user
- Không có data leakage giữa các users
- Tự động cleanup khi session kết thúc

3.3.2 Dependencies

ChatService inject các dependencies sau:

- **EntityManager:** Truy cập database và metadata
- **UserRepository:** Query thông tin users
- **RoomRepository:** Query thông tin rooms và ownership
- **OkHttpClient:** Gọi Groq API
- **Gson:** Xử lý JSON

3.3.3 State Management

ChatService maintain hai state variables quan trọng:

1. **schemaContext (String):** Lưu trữ metadata của database schema
2. **conversationHistory (List<JsonObject>):** Lưu lịch sử hội thoại (max 10 messages)

3.4 Hệ Thống Bảo Mật và Phân Quyền

3.4.1 Authorization Logic

Mỗi khi nhận request, ChatService thực hiện authorization check:

3.4.1.1 Bước 1: Lấy Thông Tin User

```
var authentication = SecurityContextHolder
    .getContext().getAuthentication();
String username = authentication.getName();
User user = userRepository.findByUsername(username);
```

3.4.1.2 Bước 2: Kiểm Tra Role

```
boolean isAdmin = user.getRoles().stream()
    .anyMatch(role -> role.getName().equals("ADMIN"));
```

3.4.1.3 Bước 3: Build Restriction Context

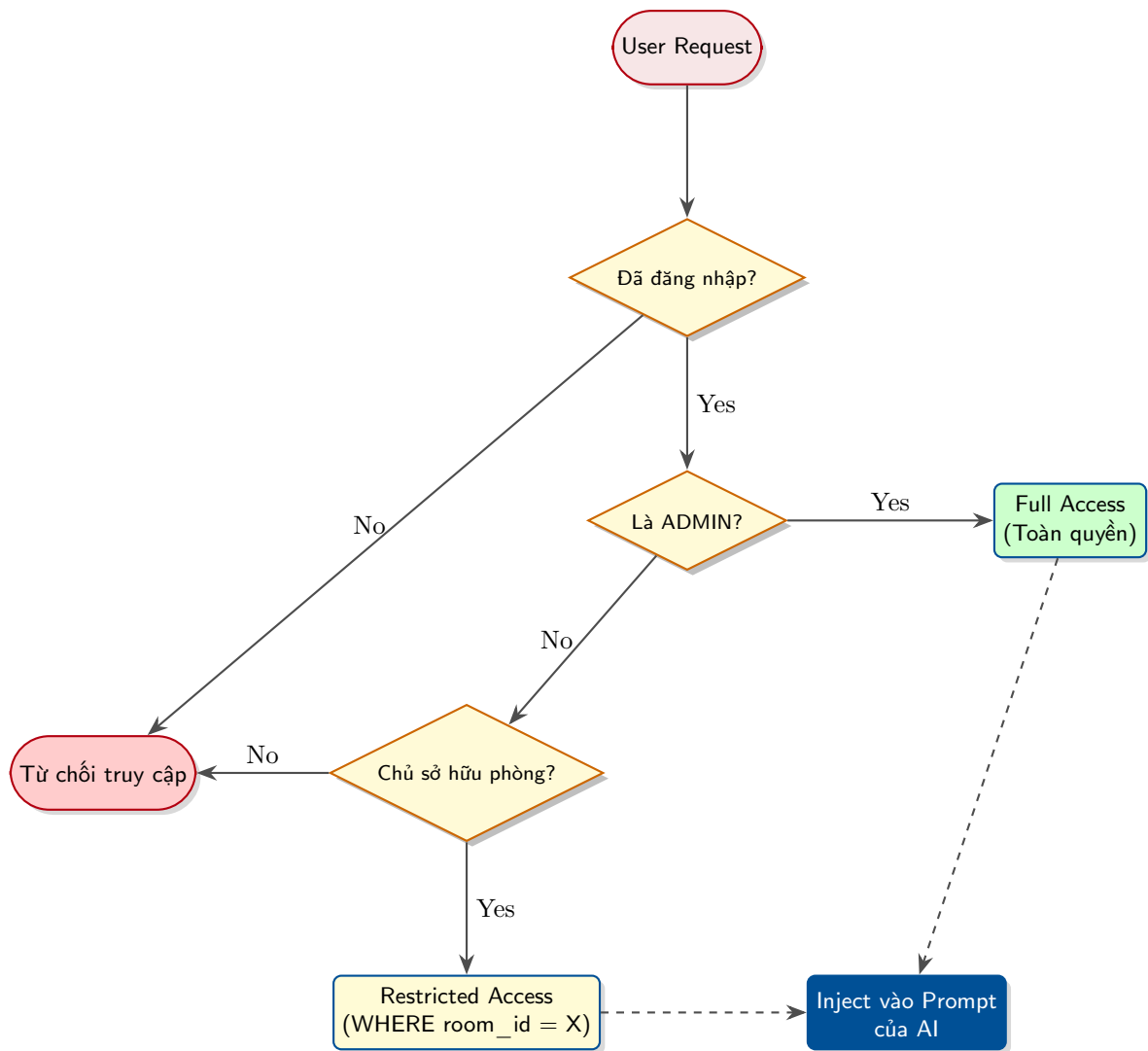
- **Nếu là ADMIN:** Không có restrictions
- **Nếu là Resident có phòng:** Restrict queries đến rooms của user
- **Nếu là Resident không có phòng:** Không cho phép query apartment data

3.4.2 Enforcement

Restriction context được inject vào AI prompts, đảm bảo SQL được sinh ra tuân thủ phân quyền ngay từ đầu.

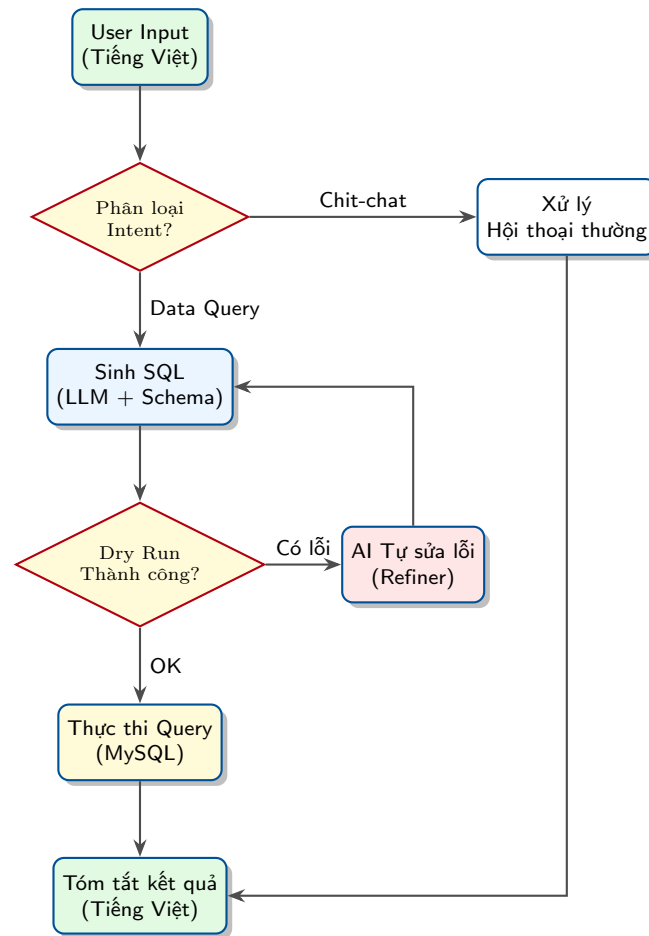
Vai trò (Role)	Quyền hạn	Cơ chế thực thi (Injection)
ADMIN	Truy cập toàn bộ	Prompt: "Bạn có quyền truy cập toàn bộ bảng."
RESIDENT	Chỉ xem phòng mình	Prompt: "Chỉ được sinh SQL có điều kiện WHERE room_id = [ID_CUA_USER]."
GUEST	Chỉ xem tin tức	Prompt: "Từ chối mọi câu hỏi liên quan đến bảng User, Room, Fee."

Bảng 3.2: Ma trận phân quyền tích hợp AI



Hình 3.1: Lưu đồ thuật toán phân quyền động (Dynamic RBAC)

3.5 Pipeline Xử Lý Tin Nhắn



Hình 3.2: Quy trình xử lý sinh SQL và tự sửa lỗi (Self-healing Pipeline)

ChatService xử lý mỗi tin nhắn qua 5 bước chính:

3.5.1 Bước 1: Authorization Check

Xác định quyền truy cập của user và build restriction context.

3.5.2 Bước 2: Intent Classification

Phân loại intent của message:

- **Chit-chat:** Greeting, thanks, casual conversation
- **Data query:** Questions về apartment data

Sử dụng AI model với temperature = 0.1 để classification chính xác.

3.5.3 Bước 3: SQL Generation

Nếu là data query, sinh SQL qua hai giai đoạn:

3.5.3.1 Giai Đoạn 1: Decomposer

- Build prompt với schema context và restrictions

- Gọi AI để decompose question thành sub-questions
- Generate SQL step-by-step
- Extract SQL từ response

3.5.3.2 Giai Đoạn 2: Refiner (nếu cần)

- Thử execute SQL (dry run)
- Nếu có lỗi, capture error details
- Gọi AI để fix SQL
- Return corrected SQL

3.5.4 Bước 4: Query Execution

- Validate SQL (phải bắt đầu với SELECT)
- Execute qua EntityManager
- Normalize result set
- Handle empty results

3.5.5 Bước 5: Result Summarization

- Limit results đến 10 rows cho summarization
- Gọi AI để tóm tắt thành natural language
- Temperature = 0.5 (balanced)
- Trả về câu trả lời bằng tiếng Việt

3.6 Nội Quan Schema Động

3.6.1 Initialization

Khi ChatService được khởi tạo, method `initSchema()` (annotated với `@PostConstruct`) tự động chạy để extract database schema.

3.6.2 Extraction Process

1. **Get Entity Metadata:** Sử dụng JPA Metamodel API
2. **Extract Table Names:** Từ `@Table` annotation hoặc class name
3. **Extract Column Names:** Từ `@Column` annotation hoặc field name
4. **Handle Enums:** Liệt kê tất cả enum values
5. **Handle Relationships:** Identify foreign keys và collection tables

3.6.3 Helper Methods

- `getPhysicalTableName():` Convert Java class name $\rightarrow$ table name
- `getPhysicalColumnName():` Convert Java field name $\rightarrow$ column name
- `camelToSnake():` Convert camelCase $\rightarrow$ snake_case
- `getAllFields():` Get all fields including inherited ones

3.6.4 Generated Schema Context

Output là string chứa thông tin đầy đủ về schema:

```
### DATABASE SCHEMA (PHYSICAL TABLES & COLUMNS):  
- Table 'rooms':  
  + 'id' (Long)  
  + 'room_number' (String)  
  + 'people_count' (Integer)  
- Table 'vehicles':  
  + 'id' (Long)  
  + 'room_id' (Long)  
  + 'type' (VehicleType) (ENUM Values: [MOTORBIKE, CAR])
```

3.7 Quản Lý Hội Thoại

3.7.1 Conversation History

ChatService maintain rolling window của 10 messages gần nhất:

- Mỗi message là JsonObject với role (user/assistant) và content
- Automatically remove oldest message khi đạt limit
- Used để provide context cho AI model

3.7.2 Context Management

Khi gọi AI API, conversation history được include trong messages:

```
JSONArray messages = new JSONArray();  
messages.add(systemPrompt);  
messages.addAll(conversationHistory); // Context  
messages.add(currentMessage);
```

3.7.3 Lợi Ích

- Hỗ trợ follow-up questions
- Resolve pronouns và references
- Maintain conversation flow
- Better user experience

3.8 Tích Hợp AI

3.8.1 Groq API Client

ChatService sử dụng OkHttp3 client để gọi Groq API.

3.8.1.1 Configuration

- **Connect Timeout:** 60 seconds
- **Read Timeout:** 60 seconds
- **Model:** openai/gpt-oss-120b
- **API Key:** Injected từ configuration

3.8.1.2 Request Format

```
{
  "model": "openai/gpt-oss-120b",
  "temperature": 0.1,
  "messages": [
    {"role": "system", "content": "..."},
    {"role": "user", "content": "..."}
  ]
}
```

3.8.2 Temperature Settings

Temperature được điều chỉnh theo từng task:

Task	Temperature	Lý do
SQL Generation	0.1	Cần deterministic, chính xác
Intent Classification	0.1	Cần precise classification
Chit-chat	0.7	Cần natural, conversational
Summarization	0.5	Cân bằng accuracy và naturalness

Bảng 3.3: Temperature settings theo task

3.9 Semantic Mapping

3.9.1 Vietnamese-to-SQL Mapping

ChatService định nghĩa mapping rules cho các terms tiếng Việt:

Tiếng Việt	SQL Mapping
xe máy	type = 'MOTORBIKE'
ô tô	type = 'CAR'
số người	people_count
số phòng	room_number
phí	fees table
đóng góp	contributions table

Bảng 3.4: Semantic mapping rules

3.9.2 Evidence String

Mapping rules được inject vào AI prompts dưới dạng "evidence":

```
### SEMANTIC MAPPING RULES (Vietnamese -> DB):
- 'xe máy' -> 'type' = 'MOTORBIKE'
- 'ô tô' -> 'type' = 'CAR'
- 'số người' -> 'people_count'
- 'số phòng' -> 'room_number'
```

3.10 Xử Lý Lỗi

3.10.1 Error Types

ChatService xử lý nhiều loại lỗi khác nhau:

1. **SQL Syntax Error:** Tự động retry với self-correction
2. **Authorization Error:** Return "Bạn không có quyền..."
3. **Invalid Query:** Return "Câu hỏi không liên quan..."
4. **Empty Results:** Return "Không tìm thấy dữ liệu..."
5. **System Error:** Return "Lỗi hệ thống..."

3.10.2 Error Recovery

- SQL errors trigger automatic correction via AI
- Errors được logged cho debugging
- User-friendly messages cho end users
- Không expose technical details

3.11 Performance Optimization

3.11.1 Schema Caching

Schema được extract một lần khi service initialization, không phải repeat mỗi request.

3.11.2 Result Limiting

Chỉ lấy 10 rows đầu tiên cho summarization, prevent large payloads.

3.11.3 Connection Pooling

OkHttpClient reuse connections, support HTTP/2 multiplexing.

3.11.4 Conversation Window

Limit history đến 10 messages, prevent token limit issues và reduce API latency.

3.12 Configuration Management

3.12.1 External Configuration

ChatService sử dụng @Value annotation để inject configuration:

```
@Value("${groq.api.key}")
private String apiKey;
```

3.12.2 Application Properties

Configuration được define trong application.yaml:

```
groq:
  api:
    key: ${GROQ_API_KEY}

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/apartment_db
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}
```

3.12.3 Environment Variables

Sensitive values được inject qua environment variables:

- GROQ_API_KEY
- DB_USERNAME
- DB_PASSWORD

3.13 Data Transfer Objects

3.13.1 ChatRequestDto

Đại diện cho request từ client:

```
public class ChatRequestDto {  
    private String message;  
    // getters and setters  
}
```

3.13.2 ChatResponse

Đại diện cho response trả về client:

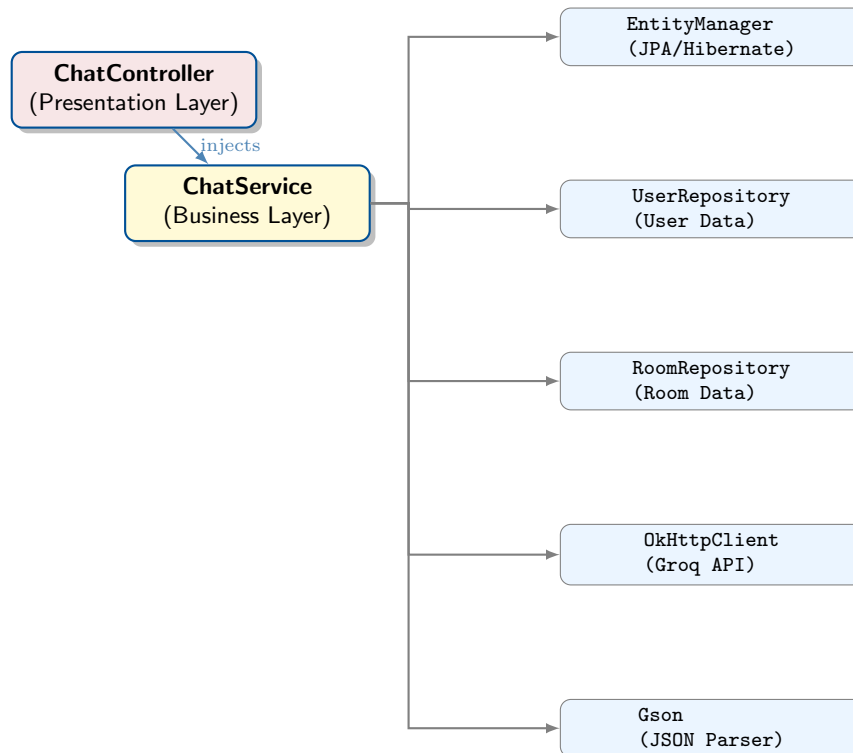
```
public class ChatResponse {  
    private String response;  
    // getters and setters  
}
```

3.14 Tương Tác Giữa Các Thành Phần

3.14.1 Request Flow

1. Client gửi POST request đến ChatController
2. ChatController parse request thành ChatRequestDto
3. ChatController gọi chatService.processMessage()
4. ChatService thực hiện:
 - Authorization check
 - Intent classification
 - SQL generation (nếu cần)
 - Query execution
 - Result summarization
5. ChatService return ChatResponse
6. ChatController serialize response thành JSON
7. JSON được gửi về client

3.14.2 Component Dependencies



Hình 3.3: Sơ đồ cây phụ thuộc (Dependency Injection Tree) trong ChatService

3.15 Tóm Tắt

Hệ thống ApartmentAI bao gồm các thành phần chính:

- **ChatController:** REST API layer, handle HTTP communication
- **ChatService:** Core business logic, session-scoped
- **Authorization System:** RBAC with dynamic restrictions
- **SQL Generation Pipeline:** Two-stage với self-correction
- **Schema Introspection:** Dynamic metadata extraction
- **Conversation Management:** Rolling window history
- **AI Integration:** Groq API với optimized temperatures
- **Semantic Mapping:** Vietnamese-to-SQL translations

Các thành phần này phối hợp chặt chẽ để tạo nên một hệ thống chatbot thông minh, bảo mật và user-friendly.

Chương 4

Cài đặt chương trình

4.1 Cấu trúc dự án

4.1.1 Backend (Spring Boot)

Cấu trúc mã nguồn Backend được tổ chức theo mô hình Layer (Lớp) tiêu chuẩn trong Spring Boot tại gói 'com.blumoonproject':

- **controller/**: Chứa các lớp 'RestController' (như 'UserController', 'ChatController') để định nghĩa các API endpoints.
- **service/**: Chứa các lớp Service (như 'UserService', 'ChatService') thực thi logic nghiệp vụ.
- **repository/**: Chứa các Interface kế thừa 'JpaRepository' để truy xuất dữ liệu từ MySQL.
- **entity/**: Chứa các Java Class ánh xạ tới bảng trong Database (JPA Entities).
- **dto/**: Data Transfer Objects, dùng để trao đổi dữ liệu giữa Client và Server.
- **configuration/**: Chứa các cấu hình về Security, CORS, Swagger.

4.1.2 Frontend (React)

Dự án Frontend được cấu trúc trong thư mục 'src' như sau:

- **components/**: Chứa các thành phần giao diện tái sử dụng (Button, Navbar, Chatbot).
- **pages/**: Chứa các trang màn hình chính (Login, Home, Dashboard).
- **context/**: Chứa Global State (AuthContext) để quản lý trạng thái đăng nhập toàn ứng dụng.
- **routes/**: Định nghĩa các luồng điều hướng và bảo vệ Route (ProtectedRoute).

4.2 Cài đặt tính năng Chatbot AI

Đây là tính năng nổi bật của dự án, cho phép người dùng "chat" với cơ sở dữ liệu.

4.2.1 Logic xử lý (ChatService.java)

Backend sử dụng kỹ thuật RAG đơn giản để cung cấp ngữ cảnh Database cho mô hình ngôn ngữ lớn (LLM). Quy trình xử lý như sau:

1. **Khởi tạo Schema Context**: Khi ứng dụng khởi động, hệ thống tự động quét các Entity (Bảng) trong Database để tạo ra một chuỗi văn bản mô tả cấu trúc dữ liệu (Tên bảng, tên cột).
2. **Tiếp nhận câu hỏi**: Khi người dùng gửi câu hỏi (ví dụ: "Cho tôi xem phí tháng này"), 'ChatController' nhận request và chuyển sang 'ChatService'.

3. **Gọi API AI (Groq):** Hệ thống gửi một prompt đến API Groq (model 'openai/gpt-oss-120b'). Prompt bao gồm:

- Mô tả cấu trúc Database (Schema Context).
- Câu hỏi của người dùng.
- Yêu cầu AI trả lời dựa trên dữ liệu hoặc tạo câu truy vấn SQL (nếu cần).

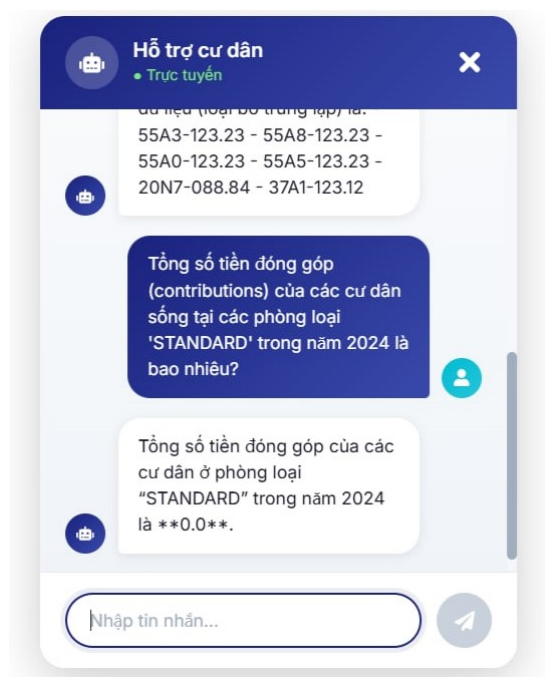
4. **Trả về kết quả:** AI phân tích và trả về câu trả lời tự nhiên cho người dùng.

```
1 @Service
2 public class ChatService {
3     // ...
4     @Value("${groq.api.key}")
5     private String apiKey;
6
7     private static final String API_URL = "https://api.groq.com/openai/v1/chat/
8         completions";
9
10    public ChatResponse processMessage(ChatRequestDto request) {
11        // Xây dựng message gửi tới LLM kèm Database Schema
12        // ...
13        // Gọi OkHttpClient thực request
14        // ...
15    }
```

Listing 4.1: Trích đoạn ChatService.java

4.2.2 Giao diện Chatbot (Chatbot.js)

Giao diện Chatbot được thiết kế dạng Floating Widget (hộp thoại nổi) ở góc màn hình. Sử dụng React State để lưu lịch sử hội thoại và 'fetch' API tới Backend.



Chương 5

Đánh Giá và Phân Tích Kết Quả

Sau quá trình xây dựng và thử nghiệm, nhóm thực hiện đã tiến hành đánh giá hệ thống ApartmentAI dựa trên các tiêu chí về chức năng, hiệu năng và tính bảo mật.

5.1 Phân Tích Điểm Mạnh - Điểm Yếu

Dưới đây là bảng tổng hợp các ưu điểm và hạn chế của hệ thống hiện tại:

Điểm Mạnh (Strengths)	Hạn Chế (Weaknesses)
• Hiểu Tiếng Việt tốt: Xử lý được ngôn ngữ tự nhiên tiếng Việt đa dạng, không cần tuân theo cú pháp cứng nhắc.	• Độ trễ (Latency): Do phải gọi API bên thứ 3 (Groq), thời gian phản hồi còn chậm (2-4s).
• Tự động sửa lỗi (Self-healing): Khả năng tự phát hiện lỗi cú pháp SQL và yêu cầu AI sửa lại mà không cần người dùng can thiệp.	• Phụ thuộc API: Hệ thống sẽ ngừng hoạt động nếu Groq Cloud bảo trì hoặc mất kết nối mạng.
• Bảo mật động: Cơ chế tiêm luật phân quyền vào prompt giúp ngăn chặn rò rỉ dữ liệu ngay từ bước sinh câu lệnh.	• Giới hạn ngữ cảnh: Nếu Database quá lớn, schema có thể vượt quá giới hạn token của LLM.
• Triển khai dễ dàng: Kiến trúc Docker hóa giúp dễ dàng cài đặt trên mọi môi trường.	• Chỉ hỗ trợ SELECT: Chưa hỗ trợ các tác vụ ghi (Insert/Update) qua chat vì lý do an toàn.

Bảng 5.1: Phân tích ưu nhược điểm của hệ thống

5.2 Kết Quả Đạt Được

5.2.1 Về mặt Chức năng

- ✓ Hỗ trợ cư dân tra cứu thông tin phòng, xe, phí dịch vụ bằng câu hỏi thường ngày.
- ✓ Hỗ trợ Ban quản lý thống kê dữ liệu nhanh chóng (Vd: "Tổng phí thu được tháng này?").
- ✓ Tích hợp trôi chảy vào giao diện Web hiện có.

5.2.2 Về mặt Kỹ thuật

- ✓ Xây dựng thành công pipeline RAG (Retrieval-Augmented Generation) cơ bản cho dữ liệu có cấu trúc.
- ✓ Ứng dụng thành công kỹ thuật Prompt Engineering (Few-shot, CoT) để nâng cao độ chính xác.
- ✓ Quản lý Schema động: Tự động cập nhật ngữ cảnh cho AI khi cấu trúc Database thay đổi.

Chương 6

Kết luận và Hướng phát triển

6.1 Kết luận

Qua quá trình thực hiện đồ án "Xây dựng hệ thống quản lý chung cư Apartment-AI", nhóm thực hiện đã đạt được những kết quả quan trọng sau:

- Xây dựng thành công quy trình nghiệp vụ quản lý chung cư trọn vẹn từ quản lý cư dân, tính toán phí đến xử lý sự cố.
- Ứng dụng thành công các công nghệ tiên tiến nhất hiện nay là Java Spring Boot (Backend) và React 19 (Frontend).
- Đặc biệt, nhóm đã nghiên cứu và tích hợp thành công mô hình ngôn ngữ lớn (LLM) vào bài toán quản lý dữ liệu doanh nghiệp, tạo ra một trợ lý ảo hữu ích cho cư dân.

6.2 Hạn chế

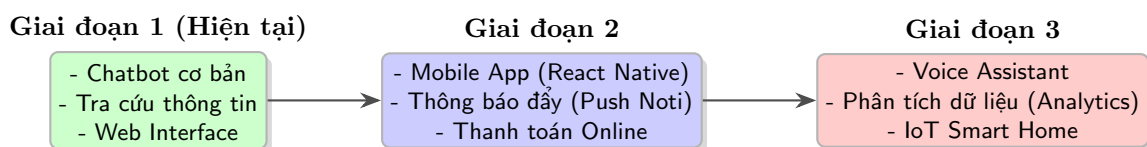
Bên cạnh những kết quả đạt được, hệ thống vẫn còn một số hạn chế:

- Chatbot hiện tại phụ thuộc vào API bên thứ ba (Groq), cần kết nối Internet ổn định.
- Chưa có phiên bản Mobile App native (iOS/Android).

6.3 Hướng phát triển

Trong tương lai, nhóm dự kiến sẽ phát triển thêm các tính năng:

- Phát triển ứng dụng di động để gửi thông báo đẩy (Push Notification) tiện lợi hơn.
- Tích hợp cổng thanh toán trực tuyến (VNPay, Momo) để cư dân đóng phí ngay trên ứng dụng.
- Nâng cấp AI để có thể thực hiện các tác vụ chủ động như nhắc đóng phí tự động hoặc phát hiện bất thường trong dữ liệu tiêu thụ điện nước.



Hình 6.1: Lộ trình phát triển sản phẩm (Roadmap)

6.4 Phân công nhiệm vụ

Dưới đây là bảng tổng hợp các nhiệm vụ chính được phân công cho từng thành viên trong quá trình thực hiện đồ án:

Sinh viên thực hiện	Nhiệm vụ phụ trách
Nguyễn Quang Huy	• Xây dựng Frontend (ReactJS) và Backend (Spring Boot). • Thiết kế Database, API quản lý cư dân và xác thực (Auth). • Cấu hình Docker và triển khai hệ thống (Deployment).
Mai Lê Phú Quang	• Phát triển ChatService và tích hợp mô hình ngôn ngữ lớn (LLM). • Xây dựng thuật toán Text-to-SQL và kỹ thuật Prompt Engineering. • Xử lý logic ngữ cảnh hội thoại và cơ chế tự sửa lỗi SQL.

Bảng 6.1: Bảng phân công nhiệm vụ thành viên

Tài liệu tham khảo

- [1] VMware, Inc. (2024). *Spring Boot Reference Documentation*. <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- [2] Bauer, C., King, G., & Gregory, G. (2015). *Java Persistence with Hibernate*. Manning Publications.
- [3] Spring.io. (2024). *Spring Security Architecture*. <https://spring.io/guides/topicals/spring-security-architecture>
- [4] Vaswani, A., et al. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems.
- [5] Zhong, V., Xiong, C., & Socher, R. (2017). *Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning*. arXiv preprint arXiv:1709.00103.
- [6] Yu, T., et al. (2018). *Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task*. EMNLP 2018.
- [7] Wei, J., et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. Advances in Neural Information Processing Systems.
- [8] Groq, Inc. (2024). *GroqCloud API Documentation*. <https://console.groq.com/docs/api-reference>
- [9] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine.